# What I Learned: Aurora Educators Program, Robotic Arms Track

**Name:** [Your Name] **Program:** Aurora Educators Program, Robotic Arms Track **Capstone:** Autonomous and Manual Robotic Arm Simulation

## Overview

This program took me from asking "what is a robot?" to building a simulated robotic arm that works in two modes. Along the way I learned how a robot's brain, code, muscles and senses fit together, and I used all of them in my capstone.

## The Big Idea: Sense, Decide, Act

A robot **senses** its surroundings, **decides** what to do, and **acts** on the world. A robotic arm is a robot built to reach, grip and place things. Sensors report what is happening, the controller runs the program, and actuators such as servos do the moving. Almost everything I built follows this loop.

## Microcontrollers and Arduino

- A **microcontroller** is a whole computer on one chip, with a processor, memory and input/output pins. It is the brain of the robot.
- **Arduino** is a platform made of a board (such as the Uno), the chip on it (the ATmega328P), external hardware, and the free Arduino IDE.
- A digital pin is either **HIGH (5V, on)** or **LOW (0V, off)**.
- The workflow I followed was to simulate in Tinkercad first, then build on hardware, then commit the project to GitHub.

## Programming Basics

- A **sketch** is an Arduino program. Every sketch has `setup()`, which runs once, and `loop()`, which runs forever.
- **Variables and data types** (`int`, `long`, `float`, `bool`) store values, and constants lock them in place.

- `digitalWrite()` drives outputs and `digitalRead()` reads inputs. With `INPUT_PULLUP`, a pressed button reads **LOW**.
- `if/else` logic lets the robot make decisions, and `delay()` lets it sequence actions.
- The **Serial Monitor** shows what the robot is thinking, which makes it the best debugging tool.
- I practised with Blink, a button controlling an LED, and a traffic light controller.

## Servos and Motion

- An **actuator** turns the controller's decisions into movement. A servo is a closed-loop system: a motor, gears, a potentiometer and a control board that keep checking the real angle against the target.
- `servo.write(angle)` sets an angle from 0 to 180 degrees. Servos use a **50 Hz** signal, and the **pulse width** (about 1 to 2 ms) sets the angle.
- The **read, map, write** pattern lets a potentiometer control a servo: `analogRead()`, then `map()`, then `write()`.
- Servos draw more current than the Arduino should supply, so they need proper power with a shared ground.
- A full arm has several joints (base, shoulder, elbow, wrist, gripper), and the joints that fight gravity need stronger servos.

## Sensors and Communication

- The **HC-SR04** ultrasonic sensor sends a sound pulse and times the echo.
- **Distance (cm) = echo time (microseconds) x 0.034 / 2**. I divide by 2 because the sound travels to the object and back.
- Sensors are not perfect, so the code should handle bad readings, such as no echo, instead of trusting every number.
- **Serial communication** sends data one bit at a time, and both sides must use the same baud rate.
- A **Bluetooth module (HC-05)** acts as a wireless serial link, and a **command protocol** gives meaning to the characters sent, such as "move joint 1 to 90 degrees".

## Projects and Assignments I Built

Each project practised a new skill and led up to the capstone. All were simulated in Tinkercad Circuits on an Arduino Uno.

| Project | What I built | What I practised |
|---|---|---|
| LED basics | LED blink and a button-controlled LED | Digital output and input, `INPUT_PULLUP` |
| Traffic light | Three LEDs cycling red, green, yellow | Sequencing with `delay()` |
| Assignment A | A 6-degree-of-freedom robot arm with six servos, each controlled by its own potentiometer | Servo control and the read, map, write pattern |
| Assignment B | A distance alert system with an ultrasonic sensor, a servo and a buzzer | Sensors, and finding and fixing a weakness in my own code |
| Assignment C | An automatic dustbin lid that opens when a hand comes near, with LEDs and a hold-open button | Combining a sensor, a servo and a button into one working system |

## Capstone: Autonomous and Manual Robotic Arm

The capstone used a microcontroller, an ultrasonic sensor, a buzzer, potentiometers and servo motors. I built it in Tinkercad, with a push button to switch modes.

| Capstone feature | What I used |
|---|---|
| Program structure and mode button | `setup()`, `loop()`, `INPUT_PULLUP` |
| Debugging modes and distances | Serial Monitor |
| Moving the base, shoulder, elbow and gripper smoothly | Servo control |
| Manual control | Potentiometers mapped to servo angles |
| Detecting objects | Ultrasonic distance measurement |

| Capstone feature | What I used |
| --- | --- |
| Switching behaviour | A state variable for manual and autonomous mode |

**Manual mode:** one potentiometer rotates the base and another extends the arm. The buzzer warns me when an object gets within 8 cm.

**Autonomous mode:** the base sweeps while the sensor scans. When an object is within 20 cm, the arm picks it up, moves it to a drop zone, releases it, and returns home.

## Skills I Gained

- Wiring circuits and reading pin diagrams
- Writing structured Arduino code with functions and state
- Controlling servos smoothly
- Measuring distance and handling bad sensor readings
- Debugging with the Serial Monitor and fixing real errors, such as an incomplete code paste and mixing up the 3-pin and 4-pin sensors
- Documenting a project with a README and sharing it on GitHub

## Key Takeaway

Every robot follows the same loop: **sense, decide, act**. This program let me build that loop from scratch, and the capstone put it all together.